DEVELOPER TOOLS

AI · ML · DS / REVISION NOTES

YOUTUBE · TECH WITH TIM

Go Programming — Full Course

Variables, types, control flow, functions, structs and more

2 SECTIONS

5 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

6 Different Programming Languages 3

7 Interpreters and Compilers 3

LECTURE 10 5

1. Go (Golang) was designed at Google.
2. It was designed to have high performance, yet simple and easy to understand syntax. Means maintain the performance like C/C++ but also easy to understand and write.
3. Go is a statically typed compiled language and useful for running backend services. Go is primarily used for backend. So, you can't use it to build user interface or entire websites. It is mainly a backend language.
4. It's also great for networking and multiprocessing.
5. Go Use cases:

- Cloud and Network Services:
- Command Line Interfaces(CLIs)
- Backend Web development (Creating APIs, Authentication Services etc)
- Automation and DevOps
- Utilities and Standalone Tools

TOPIC 6

Different Programming Languages

- There are various programming languages, each of which has its own use cases and language features.
- One of the main feature of languages we concern ourselves with is static and dynamic typing.
- We also care about how languages are compiled and interpreted.

Javascript is good for front-end web development where we are building user interfaces whereas Go is good for backend web development where we are building APIs, authentication services etc.

There is no one good programming language. Each programming language has its own use cases and language features.

TOPIC 7

Interpreters and Compilers

- When we(humans) write code, we write something known as "source code".
- Before this source code can be executed by our compilers, it must first be converted into a language that machine can understand.

- A compiler transforms our source code into something called byte code/machine code that can be ran directly by the CPU. So, when we say a language is a compiled language, what that means is that we take the source code, we compile it into this new type of code which is our byte code or machine code and then directly from the processor of our computer, we can execute that code.
- An interpreter transforms and executes source code one line at a time.

So, the compiler takes all of our code and gives us a new file and that new file is what we can actually execute. Interpreter converts the source code into that code that can be understand by the machine and it executes it at the same time. Some languages actually use both compiler and interpreter.

- Javascript is an interpreted language; Go is a compiled language.

Different optimizations occur at compile time. Compile time means when you are compiling the code and run time means when you are running the code. Whereas with interpreter, what happens is since we are not compiling our code, it can be a lot slower to run our language through an interpreter because we need to not only compiler transform that code into something that can be executed, but it then needs to be executed directly. That also means that a lot of times we are catching errors at runtime rather than compile time which can make our programs a little more difficult to work with when they get larger and larger.

So, to summarize this, compiled languages are typically a little bit stricter and are going to be faster to execute whereas interpreted languages are typically a more dynamic, a bit more flexible but it's often times going to run a little bit slower and catch a lot of errors at runtime which means you don't actually know you have an error until you execute the code which can be a big problem in a lot of different use cases.

Dynamic vs STatic Typing:

This is the property of all programming languages. We usually want to know that before we start learning a language or working with it now.

- Dynamic/Static typing refers to when and how the types of variables are decided.
- In dynamically typed languages, variable types are decided at run time, while in statically type languages, they are decided at compile time.

It means, when working in a language like javascript, we can change the type of a variable. For ex. we write var x=1 then later in the program we can change it to string/object or any other object. Here, the type of variable is decided when the code is running, which means we can change the types as we execute that code dynamically. Whereas in statically typed language, we actually need to define beforehand what type all of our variables are going to be. For example, if we declared x=1 then we can't change the type from integer to string or boolean later stage and if we try to change then it will give an error. Since, we know the type while compiling some code like a statically typed language and that's actually one of the reasons we are able to perform a lot of optimizations on that code.

- Javascript is a dynamically typed language whereas Go is a statically typed language.

9. Download and install golang from `https://go.dev/doc/install` and then type `go` in terminal to verify and see the output as:

```
🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/41.YouTube-Go-Programming-Full Course-Tech-with-Tim on
main [ ?1 ] 🐍 v3.14.2
╰─ go
Go is a tool for managing Go source code.

Usage:

        go <command> [arguments]

The commands are:

        bug         start a bug report
        build       compile packages and dependencies
        clean       remove object files and cached files
        doc         show documentation for package or symbol
        env         print Go environment information
        fix         apply fixes suggested by static checkers
        fmt         gofmt (reformat) package sources
        generate    generate Go files by processing source
        get         add dependencies to current module and install them
        install     compile and install packages and dependencies
        list        list packages or modules
        mod         module maintenance
        work        workspace maintenance
        run         compile and run Go program
        telemetry   manage telemetry data and settings
        test        test packages
        tool        run specified go tool
        version     print Go version
        vet         report likely mistakes in packages

Use "go help <command>" for more information about a command.

Additional help topics:

        buildconstraint build constraints
        buildjson       build -json encoding
        buildmode       build modes
        c               calling between Go and C
        cache           build and test caching
        environment     environment variables
        filetype        file types
        goauth          GOAUTH environment variable
        go.mod          the go.mod file
        gopath          GOPATH environment variable
        goproxy         module proxy protocol
        importpath      import path syntax
        modules         modules, module versions, and more
        module-auth     module authentication using go.sum
        packages        package lists and patterns
        private         configuration for downloading non-public code
        testflag        testing flags
        testfunc        testing functions
        vcs             controlling version control with GOVCS

Use "go help <topic>" for more information about that topic.


╭─🍎 ankit@MacBook-Air 💻 …/AI-ML-DS/41.YouTube-Go-Programming-Full Course-Tech-with-Tim on
main [ ?1 ] 🐍 v3.14.2
╰─
```

Lecture 10